Breaking the Sorting Barrier for Directed Single-Source Shortest Paths

Raed Abbas
Sotto la supervisione del Prof. Alessandro Barenghi
Politecnico di Milano

5 marzo 2026

Introduzione

Ingredienti

- Grafo Diretto $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ dove:
  - $\mathbf{V}$: insieme dei vertici (con $|\mathbf{V}|$=n)
  - $\mathbf{E}$: insieme dei archi (con $|\mathbf{E}|$=m)
- Lavoriamo con archi a pesi non negativi, quindi definiamo la funzione peso: $w_{uv} : \mathbf{E} \to \mathbb{R}_{\geq 0}\ \forall (u, v) \in \mathbf{E}$
- Vertice sorgente s

Obiettivo

Trovare la lunghezza del percorso più breve tra s e ogni vertice in $\mathbf{V}$.

L'approccio storico: Dijkstra Classico

La versione base seleziona il minimo scorrendo l'intero vettore $\mathbf{d}$ ad ogni iterazione.

Complessità

- **Estrazione minimo:** $\mathcal{O}(|\mathbf{V}|)$
- **Iterazioni:** $|\mathbf{V}|$
- **Complessità Computazionale:** $\mathcal{O}(|V|^2 + |E|)$

```
DIJKSTRA-NAIVE(G, w, s)
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = V
4  while Q ≠ ∅
5       u = EXTRACT-MIN-ARRAY(Q)
6       S = S ∪ {u}
7       for each vertice v ∈ G.Adj[u]
8            RELAX(u, v, w)
```

Miglioramento: DIJKSTRA con Priority Queue

L'uso di un Min-Heap (o Coda di Priorità) velocizza l'estrazione, ma introduce un costo logaritmico fisso per il *sorting* della frontiera.

Complessità

- **Estrazione minimo:** $\mathcal{O}(\log |\mathbf{V}|)$
- **Decrease Key:** $\mathcal{O}(\log |\mathbf{V}|)$
- **Complessità Computazionale:** $\mathcal{O}(|\mathbf{E}| \log |\mathbf{V}| + |\mathbf{V}| \log |\mathbf{V}|)$

(Su grafi sparsi $|\mathbf{E}| \approx |\mathbf{V}|$, il tempo è limitato a $\Omega(|\mathbf{V}| \log |\mathbf{V}|)$).

DIJKSTRA-PQ($\mathbf{G}, w, s$)

```
1  INIT-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = ∅ // Min-Heap
4  for each v ∈ V
5      INSERT (Q, v)
6  while Q ≠ ∅
7      u = EXTRACT-MIN(Q)
8      S = S ∪ {u}
9      for each v ∈ G.Adj[u]
10         RELAX-AND-DECREASE-KEY(u, v, w)
```

Operazione di Rilassamento (Relax)

Sia il classico Dijkstra che il moderno BMSSP condividono la stessa operazione fondamentale per aggiornare le distanze: il **Rilassamento** di un arco.

L'Idea Intuitiva

Immaginiamo di trovarci nel nodo u e di voler andare nel nodo v tramite l'arco (u, v).
Ci chiediamo: **"Passare da u mi permette di arrivare a v facendo meno strada?"**

La Formula

$$\textbf{se } \text{dist}(u) + w(u, v) < \text{dist}(v) \textbf{ allora } \text{dist}(v) = \text{dist}(u) + w(u, v)$$

Legenda dei termini:

- **dist**(v): La miglior distanza trovata finora dalla sorgente al nodo v.
- $w(u, v)$: Il costo (peso) dell'arco diretto che collega il nodo u al nodo v.
- $=$: Operazione di aggiornamento (assegnamento del nuovo valore).

Il Collo di Bottiglia: La Barriera del Sorting

Il Limite di Dijkstra

- L'algoritmo di Dijkstra mantiene una "frontiera" di vertici scoperti ma non ancora processati.
- Ad ogni passo, deve estrarre il vertice con la distanza minima usando una coda di priorità (Min-Heap).
- Questo continuo ordinamento richiede un tempo minimo di $\Omega(|\mathbf{V}| \log |\mathbf{V}|)$, creando un "collo di bottiglia" invalicabile sui grafi sparsi.

L'Idea del Paper

Fondere Dijkstra con l'approccio di **Bellman-Ford**: esplorare i vertici in "gruppi" delimitati da un upper-bound B, evitando di mantenere un ordine totale tra tutti i nodi scoperti.

Confronto 3 approcci

Caratteristica	Dijkstra Ingenuo	Dijkstra (Min-Heap)	BMSSP
Complessità Temporale	$\mathcal{O}(\|\mathbf{V}\|^2)$	$\mathcal{O}(\|\mathbf{E}\| + \|\mathbf{V}\| \log \|\mathbf{V}\|)$	$\mathcal{O}(\|\mathbf{E}\| \log^{2/3} \|\mathbf{V}\|)$
Complessità Spaziale	$\mathcal{O}(\|\mathbf{V}\| + \|\mathbf{E}\|)$	$\mathcal{O}(\|\mathbf{V}\| + \|\mathbf{E}\|)$	$\mathcal{O}(\|\mathbf{V}\| + \|\mathbf{E}\|)$

Introduzione al BMSSP: Assunzioni

Assunzioni

- ogni vertice v è raggiungibile da s (cioè $|\mathbf{E}| \geq |\mathbf{V}| - 1$)
  - Se violata: non succede niente, ma quest'assuzione serve matematicamente per semplificare le notazioni asintotiche.
- Modello *Comparison-Addition*: L'algoritmo lavora sui pesi degli archi usando solo operazioni di confronto e addizione.
  - **Se violata:** Se il modello permettesse manipolazioni di bit (modello *Word RAM*) e i pesi fossero limitati ai **numeri interi**, questo algoritmo non sarebbe necessario. L'algoritmo di Thorup risolve già il problema in tempo quasi-lineare sfruttando le operazioni binarie. Il BMSSP è fondamentale proprio perché garantisce questi tempi sub-ottimali operando nel modello classico con **pesi reali** ($\mathbb{R}$).
- tutte le lunghezze dei percorsi sono diverse
  - Se violata: è un'assunzione senza perdita di generalità, per evitare la formazione dei DAG (Grafo Aciclico Diretto)

Digressione: Il funzionamento dell'Algoritmo di Thorup

Per contestualizzare il limite del BMSSP, è utile ricordare come opera l'algoritmo di Thorup, che risolve il SSSP in tempo quasi-lineare per grafi diretti.

L'Idea Chiave: Bucket Sorting su bit

- **Struttura:** Invece del Min-Heap, distribuisce i vertici in **bucket** (secchi) basandosi sui bit più significativi delle distanze.
- **Operazioni Word RAM:** Utilizza operazioni a livello di bit (*bitwise shift*, identificazione del bit più significativo) che la CPU esegue in tempo costante $\mathcal{O}(1)$.
- **Complessità:** Raggiunge tempi sub-ottimali ($\mathcal{O}(|E| + |V| \log\log |V|)$) abbattendo il costo di estrazione del minimo.

Thorup: Operatività sui Bit

È fondamentale capire **come** Thorup usa i bit per aggirare l'ordinamento in tempo $\mathcal{O}(1)$.

Calcolo dell'Indice del Bucket (XOR e MSB)

- Invece di un Min-Heap, i vertici sono smistati in **bucket** indicizzati.
- Quando l'arco (u, v) viene rilassato, in quale bucket va spostato il nodo v?
- **La soluzione:** L'indice si calcola trovando il bit più significativo (*MSB*) in cui la vecchia e la nuova distanza differiscono.
  Operativamente: $\textbf{Indice} = \textbf{MSB}(\mathbf{d}_{\textbf{old}}[v] \oplus \mathbf{d}_{\textbf{new}}[v])$
- L'operatore logico *XOR* ($\oplus$) e la ricerca dell'MSB sono eseguiti in un singolo ciclo di clock della CPU nel modello *Word RAM*.

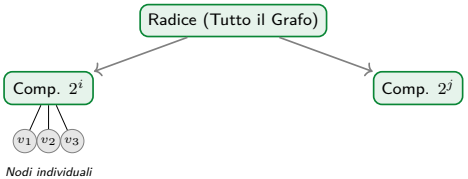
Il limite: Questo trucco esige che i pesi siano **numeri interi** (sequenze fisse di bit). Il BMSSP usa pesi **reali** ($\mathbb{R}$), sui quali operatori a livello di bit come l'XOR non hanno senso o non sono applicabili.

Thorup: La Gerarchia dei Componenti

Oltre ai bit, il secondo pilastro di Thorup è la gestione gerarchica del grafo per evitare l'ordinamento globale dei nodi.

L'approccio Hierarchical

- **Decomposizione:** Il grafo viene suddiviso in componenti in base ai pesi degli archi.
- **Component Tree:** Si crea un albero: le foglie sono i nodi, i rami sono i gruppi.
- **Visita Topologica:** Si visita l'albero in ordine topologico, non con un Min-Heap.



Risultato: Si estraggono "lotti" di nodi già pronti per il rilassamento, riducendo i confronti.

Thorup Running Example: 1. Lo scenario

Consideriamo un nodo v con una distanza provvisoria dalla sorgente.

- **Distanza corrente:** $d_{old}(v) = 13$ (binario: 1101)
- **Nuovo arco:** Arriva un arco che abbassa la distanza a $d_{new}(v) = 9$ (binario: 1001)
- **Il Problema:** In Dijkstra dovremmo riordinare la coda. Thorup deve solo decidere in quale "secchio" (*bucket*) spostare v.

Modello Word RAM

La CPU vede i pesi come stringhe di bit, permettendo calcoli logici istantanei che ignorano il valore decimale.

Thorup Running Example: 2. Il calcolo del Bucket

Per trovare il bucket corretto senza confronti, Thorup calcola la differenza bit a bit.

Calcolo XOR e MSB

$d_{old}(v) = 13$	$\rightarrow$	1 1 0 1
$d_{new}(v) = 9$	$\rightarrow$	1 0 0 1
XOR ($\oplus$)	$\rightarrow$	0 $\underline{1}$ 0 0

- **Risultato:** Il primo bit che differisce è il secondo da sinistra (valore posizionale 2^2).
- **Azione:** Il nodo v viene spostato nel bucket di livello 2.
- **Efficienza:** Questa operazione è eseguita dalla CPU in un istante ($\mathcal{O}(1)$), indipendentemente da quanti altri nodi ci siano nel sistema.

Confronto tra BMSSP e l'Algoritmo di Thorup

Perché il BMSSP è rivoluzionario se Thorup (1999) aveva già risolto il SSSP in tempo quasi-lineare? La differenza risiede nel **modello computazionale**.

Caratteristica	Algoritmo di Thorup	Algoritmo BMSSP
Modello	Word RAM	Comparison-Addition
Dominio dei Pesi	Numeri **Interi** ($\mathbb{N}$)	Numeri **Reali** ($\mathbb{R}_{\geq 0}$)
Operazioni Ammesse	Somme, Confronti, **Shift di bit, operatori logici**	Esclusivamente somme e confronti ($+$, $<$)
Logica di estrazione	*Bucket Sort* sui prefissi binari	*Batch Prepend* su Red-Black Tree

Thorup accelera l'esplorazione guardando la rappresentazione binaria dei pesi interi in memoria. Il BMSSP, invece, tratta i pesi reali come "scatole chiuse".

Introduzione al BMSSP: Notazioni

Notazioni

- $d(u)$: lunghezza del shortest path da s a u
- $\hat{d}[u]$: variabile globale per stimare $d(u)$, che viene inizializzata $\hat{d}[s] = 0$ e $\hat{d}[v] = \infty$ per i vertici che non sono la sorgente s
- u è detto complete: se $\hat{d}[u] = d(u)$ (altrimenti è detto incomplete)
- S è un set complete: se tutti i suoi vertici lo sono.
- $Pred[v]$: predecessore di v
- $\langle l, \alpha, v_\alpha, v_{\alpha-1}, ..., v_1 \rangle$: percorso di lunghezza l, che collega $v_1 = s$ con v_α attraversando α vertici

L'Intuizione: Ridurre la Frontiera

Invece di ordinare una frontiera enorme di grandezza $\Theta(|\mathbf{V}|)$, l'algoritmo restringe la ricerca a un piccolo set di **Pivot**.



- Se un percorso richiede meno di k step, viene risolto subito con Bellman-Ford.
- I nodi rimanenti dipendono da pochi **Pivot** (le radici degli alberi dei cammini minimi di profondità $\geq k$).

Rilassamento

$$\textbf{se } \mathbf{d}[u] + w_{uv} < \mathbf{d}[v] \textbf{ allora } \mathbf{d}[v] = \mathbf{d}[u] + w_{uv}$$

Bounded Multi-Source Shortest Path (BMSSP)

Il cuore dell'innovazione è la procedura ricorsiva **BMSSP(I, B, S)**:
Introduciamo prima il concetto di livello l:

- **Cos'è l?** È l'**indice del livello di ricorsione**.
  Parte da un valore massimo e decresce unitariamente ad ogni chiamata interna.
- **Controllo della Granularità:** Il livello l determina l'ampiezza massima dei blocchi ($M = 2^{(l-1)t}$) all'interno della struttura dati $\mathbf{D}$.
  - **Alti livelli di l:** Blocchi molto grandi. Bassa precisione di ordinamento, ma massima velocità per l'inserimento a lotti (*Batch Prepend*).
  - **Bassi livelli di l:** Blocchi sempre più piccoli e precisi.
- **Caso Base ($l = 0$):** L'ampiezza del blocco collassa. La ricorsione termina e l'algoritmo ripiega su un mini-Dijkstra esatto limitato a pochi vertici.

Bounded Multi-Source Shortest Path (BMSSP)

Divide et Impera

- **Input:** un livello l, un upper bound B, e un set di sorgenti S.
- **Logica:** L'algoritmo partiziona adattivamente il problema in sotto-problemi di dimensioni inferiori.
- **Ricorsione:** Scende fino al caso base ($l = 0$) dove esegue un "mini-Dijkstra" limitato all'esplorazione di $k + 1$ nodi.
- **Output:** Trova tutte le distanze reali inferiori a un nuovo bound $B' \leq B$.

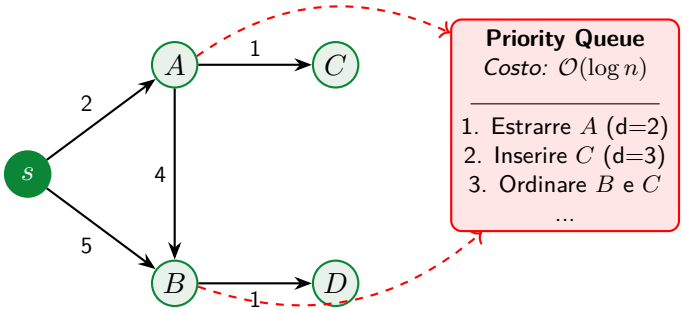
La Struttura Dati Ibrida (Lemma 3.3)

Per gestire i partizionamenti in modo efficiente, il paper introduce una struttura dati ad-hoc $\mathbf{D}$ composta da:

- **Blocchi (Liste) $\mathbf{D}_l$:** I nodi con distanze simili vengono raggruppati in blocchi di dimensione massima M. (chiamata $\mathbf{D}_0$ nel paper)
- **Albero Red-Black $\mathbf{D}_{rb}$:** Mantiene i limiti superiori (*upper bounds*) di ciascun blocco. (chiamata $\mathbf{D}_1$ nel paper)
- **Inserimento a Lotti (*Batch Prepend*):** Invece di inserire e ordinare i nuovi nodi uno ad uno (come fa la coda di priorità in Dijkstra), i nodi con distanze simili vengono raggruppati in un unico blocco. Questo intero blocco viene inserito tutto in una volta in testa alla struttura, aggirando il costo del sorting continuo.

Esempio Visivo: L'approccio Classico (Dijkstra)

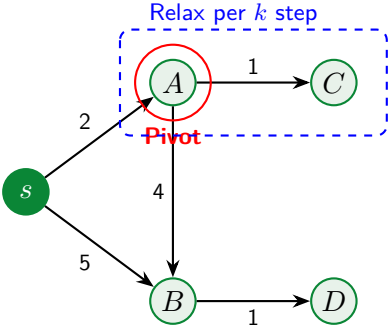
Dijkstra esplora il grafo rigorosamente in base alla distanza minima.



Problema: Anche se l'arco $A \to C$ è ovvio, Dijkstra deve **sempre** rimettere tutto in coda e riordinare l'intera frontiera.

Esempio Visivo: Il Nuovo Approccio (BMSSP)

Si usano i k step : raggruppa l'esplorazione ed evita il sorting globale.



- **Propagazione senza Priority Queue:** Si **rilassano** gli archi a cascata per k passi.
- Il nodo A genera un sotto-albero abbastanza profondo da essere promosso a **Pivot**. Solo i Pivot passano alla ricorsione successiva.

Pseudocodice del Bounded Multi-Source (BMSSP)

La procedura partiziona l'esplorazione ricorsivamente.

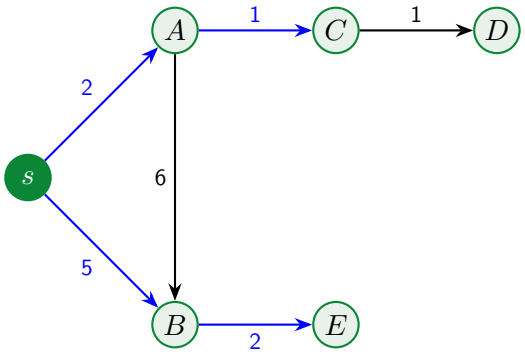
- Usa FINDPIVOTS per isolare $\mathbf{P}$.
- Estrae i lotti di minimi con $\mathbf{D}$.PULL().
- Scende di livello fino a ripiegare su un mini-Dijkstra limitato.

```
BMSSP(l, B, S)
 1  if l = 0
 2      return BASECASE(B, S)
 3  P, W = FINDPIVOTS(B, S)
 4  D.INIT(M, B)
 5  for each x ∈ P
 6      D.INSERT(⟨x, d[x]⟩)
 7  while |U| < k2^{lt} e D ≠ ∅
 8      B_i, S_i = D.PULL()
 9      B'_i, U_i = BMSSP(l − 1, B_i, S_i)
10      U = U ∪ U_i
11      K = ∅
12      for each (u, v) uscente da U_i
13          if RELAX(u, v)
14              if d[v] ∈ [B_i, B)
15                  D.INSERT(v)
16              elseif d[v] ∈ [B'_i, B_i)
17                  K = K ∪ {v}
18      D.BATCHPREPEND(K)
19  return B', U
```

Running Example: 1. Fase di FindPivots

Fissiamo la soglia $k = 2$. Iniziamo con il limite superiore $B = \infty$ e la sorgente $\mathbf{S} = \{s\}$. Il primo passo è propagare i cammini minimi provvisori per k iterazioni.

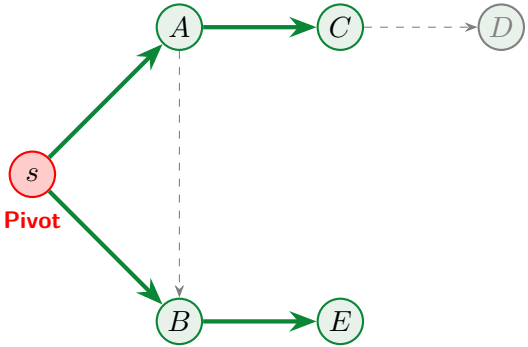
- **Step 1:** Rilassiamo da s. Troviamo A ($\mathbf{d}[A] = 2$) e B ($\mathbf{d}[B] = 5$).
- **Step 2:** Rilassiamo da A e B. Troviamo C ($\mathbf{d}[C] = 3$) ed E ($\mathbf{d}[E] = 7$).
- **Risultato:** L'insieme dei vertici scoperti è $\mathbf{W} = \{s, A, B, C, E\}$.
- **Nota:** Nessun ordinamento totale è stato effettuato.



Running Example: 2. Foresta $\mathbf{F}$ e Selezione Pivot

Costruiamo la foresta $\mathbf{F}$ analizzando i predecessori $(\mathbf{pred})$ appena scoperti in $\mathbf{W}$.

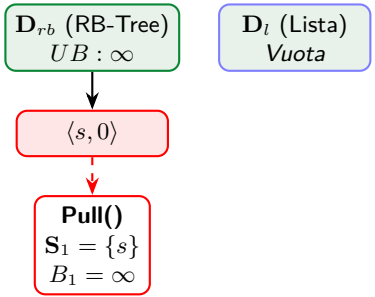
- L'albero radicato nella sorgente s ha dimensione 5.
- Poiché la dimensione è $\geq k$ $(5 \geq 2)$, il vertice s viene promosso a **Pivot**.
- Insieme dei Pivot: $\mathbf{P} = \{s\}$.
- Se un nodo genera un albero piccolo $(< k)$, non diventa Pivot e viene rimandato alla ricorsione.



Running Example: 3. Setup di $\mathbf{D}$ e Prima $\textsc{Pull}$

L'algoritmo istanzia la struttura dati ibrida $\mathbf{D}$ e vi inserisce i Pivot per prepararli alla ricorsione.

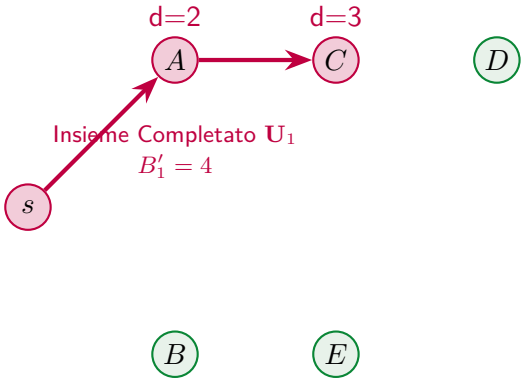
- $\mathbf{D}.\textsc{Init}(\infty)$: Si crea un RB-Tree per $\mathbf{D}_{rb}$ e una lista vuota per $\mathbf{D}_l$.
- $\mathbf{D}.\textsc{Insert}(\langle s, 0\rangle)$ inserisce la sorgente nell'albero $\mathbf{D}_{rb}$.
- **Inizio Ciclo While:**
- Eseguiamo $\mathbf{D}.\textsc{Pull}()$: estraiamo i minimi.
- **Risultato:** $\mathbf{S}_1 = \{s\}$. Poiché $\mathbf{D}$ ora è vuota, il nuovo limite temporaneo è $B_1 = \infty$.



Running Example: 4. Chiamata Ricorsiva e Ritorno

L'algoritmo invoca ricorsivamente $\textsc{Bmssp}(l-1, B_1, \mathbf{S}_1)$ per completare l'esplorazione attorno al pivot s.

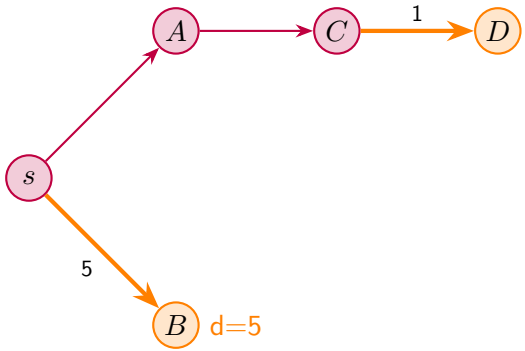
- La ricorsione scende di livello e consolida i cammini minimi locali.
- **Ritorno della Ricorsione:**
- Restituisce i vertici completati $\mathbf{U}_1 = \{s, A, C\}$.
- Restituisce un nuovo limite $B_1' = 4$ (le distanze in $\mathbf{U}_1$ sono ≤ 4).



Running Example: 5. Partizionamento dei Nuovi Nodi

Ora si esplorano gli archi **uscenti** dai nodi appena completati $\mathbf{U}_1$ e si classificano le nuove distanze in base a B_1 e B_1'.

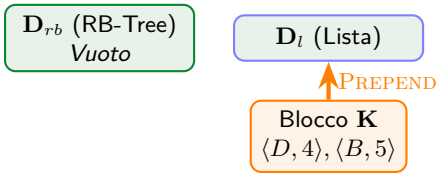
- Arco $s \xrightarrow{5} B$: Troviamo $\mathbf{d}[B] = 5$.
- Arco $C \xrightarrow{1} D$: Troviamo $\mathbf{d}[D] = 4$.
- **Controllo dei Range:**
- Ricordiamo che $B_1' = 4$ e $B_1 = \infty$.
- Entrambi i valori (4 e 5) ricadono nel range $[B_1', B_1) = [4, \infty)$.
- **Regola del Paper:** Vengono parcheggiati nell'insieme temporaneo $\mathbf{K} = \{B, D\}$.



Running Example: 6. BatchPrepend e Loop

Per non dover fare n ordinamenti in $\mathbf{D}_{rb}$, i vertici in $\mathbf{K}$ vengono convertiti in un blocco e inseriti in testa a $\mathbf{D}_l$ in tempo quasi $\mathcal{O}(1)$.

- Eseguiamo $\mathbf{D}$.BatchPrepend($\mathbf{K}$).
- $\mathbf{D}_l$ riceve un nuovo blocco contenente B e D.
- **Fine prima iterazione.**
- Al giro successivo del ciclo `while`, la funzione Pull() estrarrà il minimo proprio da $\mathbf{D}_l$ senza dover cercare nell'intero albero!



Il Segreto della Complessità

Raggruppando i vertici in $\mathbf{K}$ e saltando la coda a priorità totale, si passa da $\Omega(n \log n)$ a $\mathcal{O}(m \log^{2/3} n)$.